Java Synchronization & Locks

Race Conditions, Monitors, Locks, and Advanced Synchronizers

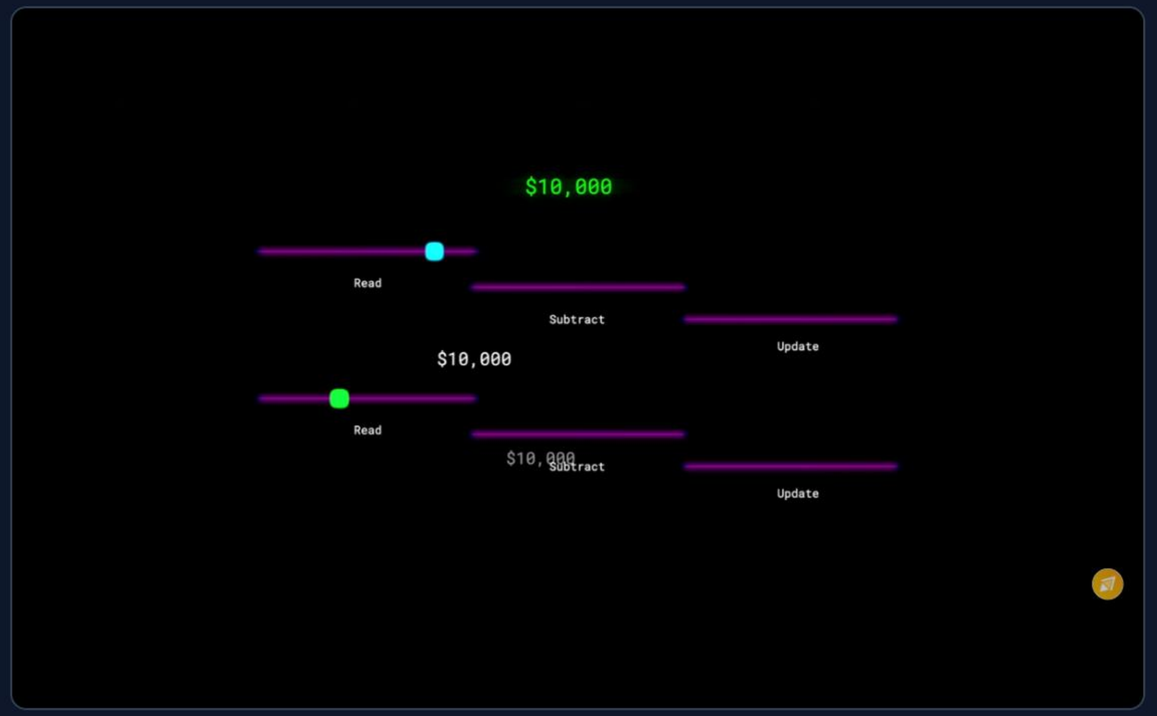
Mastering Thread Safety in High-Concurrency Environments

The Core Problem: Race Conditions

What is a Race Condition?

A race condition occurs when two or more threads access shared data and try to change it at the same time. Because the thread scheduling algorithm can swap between threads at any time, the order of execution is unpredictable.

- **Read-Modify-Write:** A common pattern (e.g., `count++`) that is not atomic. It involves reading a value, modifying it, and writing it back.
- **Lost Updates:** Thread A and Thread B read '10'. Both increment to '11'. Both write '11'. One increment is lost.
- **The Solution -> Synchronization:** Enforcing *Mutual Exclusion*. Only one thread can execute a critical section of code at a time.



Intrinsic Locks: Method vs. Block

Every object in Java has a built-in "intrinsic lock" (or monitor). We use the `synchronized` keyword to acquire it.

Synchronized Method

Locks the **entire** object (`this`) for the duration of the method. Simple, but can cause performance bottlenecks if the method is long.

```
public class BankAccount {
    private int balance = 0;

    // Thread acquires lock on 'this' instance
    public synchronized void deposit(int amt) {
        balance += amt; // Safe
    }
}
```

Synchronized Block

Locks only a specific section of code using a designated lock object. Reduces the scope of the lock, improving throughput.

```
public class BankAccount {
    private int balance = 0;
    private final Object lock = new Object();

    public void deposit(int amt) {
        // Do unlocked work here...

        synchronized(lock) { // Enter critical section
            balance += amt;
        } // Release lock
    }
}
```

Scope: Object-Level vs. Class-Level Locks

Object-Level Lock

Protects non-static data. The lock is acquired on the specific **instance** of the class. Two threads acting on *different* instances will NOT block each other.

```
public class Counter {
    private int count = 0;

    // Locks the specific instance (e.g., counter1)
    public synchronized void increment() {
        count++;
    }

    // Equivalent block format:
    public void decrement() {
        synchronized(this) {
            count--;
        }
    }
}
```

Class-Level Lock

Protects static data. The lock is acquired on the **Class object** (e.g., `Counter.class`). This enforces mutual exclusion across ALL instances globally.

```
public class Counter {
    private static int globalCount = 0;

    // Locks the Counter.class object
    public static synchronized void globalInc() {
        globalCount++;
    }

    // Equivalent block format:
    public void globalDec() {
        synchronized(Counter.class) {
            globalCount--;
        }
    }
}
```

Advanced: ReentrantLock

Part of `java.util.concurrent.locks`, `ReentrantLock` offers advanced capabilities over implicit `synchronized` blocks.

- **Reentrancy:** A thread can acquire the same lock multiple times without deadlocking itself. It keeps a hold count.
- **Interruptible:** `lockInterruptibly()` allows a waiting thread to be interrupted.
- **Timeouts:** `tryLock(time, unit)` tries to get the lock but gives up after a timeout.
- **Fairness:** Can be initialized as a "fair" lock (longest-waiting thread gets the lock next).

⚠ **Mandatory Rule:** You MUST unlock in a `finally` block to ensure the lock is released even if an exception is thrown.

```java
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

public class SafeQueue {
    // Create a fair ReentrantLock
    private final Lock lock = new ReentrantLock(true);

    public void doWork() {
        lock.lock(); // Block until lock is acquired
        try {
            // Critical Section
            System.out.println("Lock acquired by: " +
                Thread.currentThread().getName());

            // Reentrancy: Calling another method that
            // requires the SAME lock works perfectly!
            doMoreWork();
        } finally {
            lock.unlock(); // ALWAYS in a finally block
        }
    }

    public void doMoreWork() {
        lock.lock();
        try { /* Nested work */ } finally { lock.unlock(); }
    }
}
```

Shared vs. Exclusive: ReadWriteLock

Standard locks are mutually exclusive. `ReentrantReadWriteLock` splits the lock in two, optimizing read-heavy data structures (like caches).

📖 Read Lock (Shared)

Multiple threads can acquire the read lock simultaneously, as long as no thread holds the write lock. Boosts concurrency massively.

🖊 Write Lock (Exclusive)

Only ONE thread can hold the write lock. Furthermore, it can only be acquired if NO threads are currently holding the read lock.

```java
import java.util.concurrent.locks.*;

public class ConcurrentCache {
    private final ReadWriteLock rwLock =
        new ReentrantReadWriteLock();
    private String data = "Initial";

    public String readData() {
        rwLock.readLock().lock(); // Shared
        try {
            return data; // Many threads can be here
        } finally {
            rwLock.readLock().unlock();
        }
    }

    public void writeData(String newData) {
        rwLock.writeLock().lock(); // Exclusive
        try {
            data = newData; // Only ONE thread here
        } finally {
            rwLock.writeLock().unlock();
        }
    }
}
```

Beyond Locks: java.util.concurrent Synchronizers

Higher-level coordination objects for complex thread routing

Semaphore

Maintains a set of permits. Used to limit the number of threads accessing a specific resource simultaneously (e.g., Connection Pools).

CyclicBarrier

A synchronization point where a fixed number of threads must wait for each other to arrive before they can all proceed together.

Phaser

A more flexible, reusable barrier. Supports a dynamic number of threads and multiple phases of execution.

Controlling Access: Semaphore

A `Semaphore` acts like a bouncer at a club. It holds a specific number of "permits".

- **acquire():** Takes a permit. If zero permits are available, the thread blocks until one is released.
- **release():** Returns a permit, potentially waking up a blocked thread.
- **Use Case:** Rate limiting, bounding collections, database connection pools.

```
import java.util.concurrent.Semaphore;

public class ConnectionPool {
    // Only 3 threads allowed concurrently
    private final Semaphore semaphore = new Semaphore(3);

    public void useConnection() {
        try {
            semaphore.acquire(); // Get permit
            System.out.println(Thread.currentThread().getName()
                + " got connection.");

            // Simulate using connection
            Thread.sleep(2000);

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        } finally {
            System.out.println(Thread.currentThread().getName()
                + " releasing.");
            semaphore.release(); // ALWAYS release in finally
        }
    }
}
```

Rendezvous: CyclicBarrier

Allows a set of threads to all wait for each other to reach a common barrier point.

- **Initialization:** Created with the number of participating threads.
- **await():** Threads call this and block. Once the Nth thread calls `await()`, the barrier is tripped, and ALL threads are released.
- **Action:** Can run an optional `Runnable` action exactly once when the barrier trips, before threads are released.
- **Cyclic:** Can be reused after it is tripped (unlike `CountDownLatch`).

```
import java.util.concurrent.CyclicBarrier;

public class DataCruncher {
    public static void main(String[] args) {
        // Barrier for 3 threads. When all 3 arrive, run action
        CyclicBarrier barrier = new CyclicBarrier(3, () -> {
            System.out.println("All parts computed! Merging...");
        });

        for (int i = 0; i < 3; i++) {
            new Thread(() -> {
                try {
                    System.out.println("Computing part...");
                    Thread.sleep((long)(Math.random() * 1000));

                    // Arrive and wait for others
                    barrier.await();

                    System.out.println("Moving to next task.");
                } catch (Exception e) {}
            }).start();
        }
    }
}
```

Dynamic Barrier: Phaser

A `Phaser` is a more robust, flexible version of `CyclicBarrier` and `CountDownLatch`.

- **Dynamic Registration:** The number of registered parties can change over time using `register()` and `deregister()`.
- **Phases:** Execution is divided into phases. The phaser advances its phase number when all registered parties arrive.
- **Methods:**
  - `arriveAndAwaitAdvance()`: Arrive & wait.
  - `arriveAndDeregister()`: Finish & leave.

```java
import java.util.concurrent.Phaser;

public class GameLevelLoader {
    public static void main(String[] args) {
        Phaser phaser = new Phaser(1); // Register main thread

        for (int i = 0; i < 2; i++) {
            phaser.register(); // Dynamically add worker
            new Thread(() -> {
                System.out.println("Loading assets ... ");
                // Phase 0 complete for this thread
                phaser.arriveAndAwaitAdvance();

                System.out.println("Connecting to server ... ");
                // Thread is done, deregister
                phaser.arriveAndDeregister();
            }).start();
        }

        // Main thread waits for Phase 0 to complete
        phaser.arriveAndAwaitAdvance();
        System.out.println("Phase 0 (Assets) complete!");

        // Main thread deregisters, letting workers finish Phase 1
        phaser.arriveAndDeregister();
    }
}
```

Questions?

Thank you for exploring Synchronization & Locks.

```
System.exit(0);
```

Image Sources

https://i.sstatic.net/rONSe.png

Source: stackoverflow.com